---------- PANDAS ----------

Pandas is a powerful Python library used for data analysis and manipulation. It is especially useful when working with structured data like tables (similar to Excel or SQL tables).

Key Features :-

- Easy handling of tabular data
- Supports CSV, Excel, JSON, SQL
- Powerful data cleaning & transformation
- Fast operations using NumPy

1. Create Dataframe

2. Basic Dataframe Understanding

3. Save and Load data from csv

4. Rows & Columns - Selection

5. Filter Dataframe - Filtering by column values

6. Rows and Columns - Operation (Add, update, delete)

7. Working with DATE values

8. Handling Missing Values

9. Aggregation and Group By

10. Concatenate & Merge DataFrame (JOINS)

11. Broadcasting

12. Handling Duplicate Values

13. Lambda Function

In [ ]:

In [530…

```
# install pandas
! pip install pandas
```

```
Requirement already satisfied: pandas in c:\users\satya\anaconda3\lib\site-packag
es (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\satya\anaconda3\lib\site
-packages (from pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\satya\anaconda3
\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\satya\anaconda3\lib\site-
packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\satya\anaconda3\lib\sit
e-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\satya\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

In [ ]:

In [531…
```python
# import pandas
import pandas as pd
```

In [532…
```python
pd.__version__
```

Out[532…
```
'2.2.3'
```

1. Create Dataframe

It's a structured data constructed with rows and columns, similar to a SQL tables or Excel tables

In [533…
```python
# using a List
df = pd.DataFrame([11,22,33])
print(df)
```

```
    0
0  11
1  22
2  33
```

In [534…
```python
df = pd.DataFrame([11,22,33],columns=['Col_Name'])
print(df)
```

```
   Col_Name
0        11
1        22
2        33
```

In [535…
```python
# check data type
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

In [536…
```python
# using Dictionary of Lists
data={
    "Name":['Satyam','Raj','Laksh','Advik'],
    "Age":[22,25,5,10],
    "Salary":[50000, 70000,90000,60000]
}
```

In [537…
```python
df=pd.DataFrame(data)
print(df)
```

```
     Name  Age  Salary
0  Satyam   22   50000
1     Raj   25   70000
2   Laksh    5   90000
3   Advik   10   60000
```

In [538…
```python
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

2. Basic Dataframe Understanding

In [539…

```
df.head()  # top rows (by default it gives first five)
```

Out[539…

	Name	Age	Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Laksh	5	90000
3	Advik	10	60000

In [540…

```
df.head(2)
```

Out[540…

	Name	Age	Salary
0	Satyam	22	50000
1	Raj	25	70000

In [541…

```
df.tail(2)  # last row
```

Out[541…

	Name	Age	Salary
2	Laksh	5	90000
3	Advik	10	60000

In [542…

```
df.shape   # returns a tuple containing the shape of the DataFrame - rows & colu
```

Out[542…

```
(4, 3)
```

In [543…

```
# list of column names in a dataframe
df.columns
```

Out[543…

```
Index(['Name', 'Age', 'Salary'], dtype='object')
```

In [544…

```
# rename columns name
df.rename(columns={'Salary': 'Monthly_Salary'}, inplace=True)
```

In [545…

```
print(df)
```

```
     Name  Age  Monthly_Salary
0  Satyam   22           50000
1     Raj   25           70000
2   Laksh    5           90000
3   Advik   10           60000
```

In [546…

```
df.info()  # info method prints information about the DataFrame
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Name            4 non-null      object
 1   Age             4 non-null      int64
 2   Monthly_Salary  4 non-null      int64
dtypes: int64(2), object(1)
memory usage: 228.0+ bytes
```

In [547…

```
df.describe() # describe method generates descriptive statistics of DataFrame, o
```

Out[547…

	Age	Monthly_Salary
count	4.000000	4.000000
mean	15.500000	67500.000000
std	9.539392	17078.251277
min	5.000000	50000.000000
25%	8.750000	57500.000000
50%	16.000000	65000.000000
75%	22.750000	75000.000000
max	25.000000	90000.000000

3. Save and Load Data from CSV

In [548…

```
# save file - export data frame

df.to_csv('Test_data.csv',index=False)
```

In [549…

```
# load file - import dataframe

load_df= pd.read_csv('Test_data.csv')
```

In [550…

```
load_df
```

Out[550…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Laksh	5	90000
3	Advik	10	60000

4. Rows & Columns - (Selection)

In [551…

```
# select single column
df[['Name']]
```

Out[551…

	Name
0	Satyam
1	Raj
2	Laksh
3	Advik

In [552…

```
# select multiple columns
df[['Name','Monthly_Salary']]
```

Out[552…

	Name	Monthly_Salary
0	Satyam	50000
1	Raj	70000
2	Laksh	90000
3	Advik	60000

In [553…

```
# select single row || loc - label based index

df.loc[df.Name=='Raj']
```

Out[553…

	Name	Age	Monthly_Salary
1	Raj	25	70000

In [554…

```
# select multiple rows || loc - label based index

df.loc[(df.Name=='Satyam') & (df.Monthly_Salary>=50000)]
```

Out[554…

	Name	Age	Monthly_Salary
0	Satyam	22	50000

In [555…

```
df.loc[0:2]
```

Out[555…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Laksh	5	90000

In [556…

```
# Select Rows || iloc - index-value based
# df.iloc[0]

df.iloc[0:2]   # [start:stop:step]
```

Out[556…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000

5. Filter Dataframe - Filtering by column values

In [557…

```
df
```

Out[557…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Laksh	5	90000
3	Advik	10	60000

In [558…

```
# filter and store dataframe in a nwe variable

df_age_filter = df[df['Age'] >= 18]
```

In [559…

```
df_age_filter
```

Out[559…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000

In [560…

```
df[(df['Age'] >= 18) & (df['Monthly_Salary'] >= 50000)] # multiple filter condit
```

Out[560…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000

In [561…

```
# where function replace values in a DataFrame based on a condition

df.where(df['Age'] >= 18)
```

Out[561…

	Name	Age	Monthly_Salary
0	Satyam	22.0	50000.0
1	Raj	25.0	70000.0
2	NaN	NaN	NaN
3	NaN	NaN	NaN

In [562…

```
df.where(df['Age'] >= 18, other = 'Not Eligible')
```

Out[562…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Not Eligible	Not Eligible	Not Eligible
3	Not Eligible	Not Eligible	Not Eligible

6. Rows and Columns - Operation (Add, update, delete)

In [563…

```
df
```

Out[563…

	Name	Age	Monthly_Salary
0	Satyam	22	50000
1	Raj	25	70000
2	Laksh	5	90000
3	Advik	10	60000

In [564…

```
# add new column

df['Team']=['DA','DS','CEO','HR']
df
```

Out[564…

	Name	Age	Monthly_Salary	Team
0	Satyam	22	50000	DA
1	Raj	25	70000	DS
2	Laksh	5	90000	CEO
3	Advik	10	60000	HR

In [565…

```
# Add new columns using existing column/s
df['Bonus']= df['Monthly_Salary']* 0.2
df
```

Out[565…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	50000	DA	10000.0
1	Raj	25	70000	DS	14000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0

In [566…

```
# Add new row - at the end of dataframe

df.loc[len(df)] = ['ABC', 21 ,21000, 'IT', 2000]
df
```

Out[566…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	50000	DA	10000.0
1	Raj	25	70000	DS	14000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0
4	ABC	21	21000	IT	2000.0

In [567…

```
len(df)
```

Out[567…

5

In [568…

```
# update value in dataframe using index-name

df.loc[0, 'Monthly_Salary']= 100000
df
```

Out[568…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	100000	DA	10000.0
1	Raj	25	70000	DS	14000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0
4	ABC	21	21000	IT	2000.0

In [569…

```
# update value in dataframe using column-value

df.loc[df.Name == 'Raj' , 'Monthly_Salary'] = 150000
df
```

Out[569…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	100000	DA	10000.0
1	Raj	25	150000	DS	14000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0
4	ABC	21	21000	IT	2000.0

In [570…

```
# delete value - row and column value

# delete row using column-value filter
```

```
df = df.drop(df[df.Name == 'ABC'].index)   # delete row, axis = 0
df
```

Out[570…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	100000	DA	10000.0
1	Raj	25	150000	DS	14000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0

In [571…

```
# delete row using index-name filter
df.drop(1, axis=0)
```

Out[571…

	Name	Age	Monthly_Salary	Team	Bonus
0	Satyam	22	100000	DA	10000.0
2	Laksh	5	90000	CEO	18000.0
3	Advik	10	60000	HR	12000.0

In [572…

```
df.drop('Bonus' , axis=1, inplace=True)   # delete one column
df
```

Out[572…

	Name	Age	Monthly_Salary	Team
0	Satyam	22	100000	DA
1	Raj	25	150000	DS
2	Laksh	5	90000	CEO
3	Advik	10	60000	HR

In [573…

```
# delete multiple columns

# df.drop(['Bonus', 'Team'] ,axis=1, inplace = True)
```

In [574…

```
# sort values - order values in dataframe by asc or desc order

df.sort_values('Monthly_Salary')           # ascending order (by default)
```

Out[574…

	Name	Age	Monthly_Salary	Team
3	Advik	10	60000	HR
2	Laksh	5	90000	CEO
0	Satyam	22	100000	DA
1	Raj	25	150000	DS

In [575…

```
df.sort_values('Monthly_Salary', ascending = False)      # descending order
```

Out[575…

	Name	Age	Monthly_Salary	Team
1	Raj	25	150000	DS
0	Satyam	22	100000	DA
2	Laksh	5	90000	CEO
3	Advik	10	60000	HR

7. Working with DATE values

In [576…

```
# create a date column - date of joining
df['DOJ'] =['2024-01-01', '2024-01-15', '2024-03-28', '2024-03-03']  # DOJ - dat
df
```

Out[576…

	Name	Age	Monthly_Salary	Team	DOJ
0	Satyam	22	100000	DA	2024-01-01
1	Raj	25	150000	DS	2024-01-15
2	Laksh	5	90000	CEO	2024-03-28
3	Advik	10	60000	HR	2024-03-03

In [ ]:

```

```

In [577…

```
df['DOJ'] = pd.to_datetime(df['DOJ'])     # change to date-time type
```

In [578…

```
df['DOJ'].dtype  # date-time type
```

Out[578…

```
dtype('<M8[ns]')
```

NOTE :-

'<M8[ns]' in pandas is a dtype (data type) representing:

M → datetime (timestamp) , 8 → 8 bytes (64-bit) , [ns] → nanosecond precision

In [579…

```
df1 = df
```

In [580…

```
# creating a new column with incorrect date format
df1['DOJ2'] = ['01-01-2025', '15-01-2025', '28-03-2025', '03-03-2025']
df1
```

Out[580…

	Name	Age	Monthly_Salary	Team	DOJ	DOJ2
0	Satyam	22	100000	DA	2024-01-01	01-01-2025
1	Raj	25	150000	DS	2024-01-15	15-01-2025
2	Laksh	5	90000	CEO	2024-03-28	28-03-2025
3	Advik	10	60000	HR	2024-03-03	03-03-2025

In [581…
```
df1['DOJ2'].dtype  # object-type value
```

Out[581…
```
dtype('O')
```

In [582…
```
# change to date-time type using date-format
df1['DOJ2'] = pd.to_datetime(df1['DOJ2'], format = '%d-%m-%Y')
```

In [583…
```
df1['DOJ2'].dtype
```

Out[583…
```
dtype('<M8[ns]')
```

In [584…
```
df = df.drop('DOJ2', axis=1)
df
```

Out[584…

	Name	Age	Monthly_Salary	Team	DOJ
0	Satyam	22	100000	DA	2024-01-01
1	Raj	25	150000	DS	2024-01-15
2	Laksh	5	90000	CEO	2024-03-28
3	Advik	10	60000	HR	2024-03-03

In [585…
```
# extract year, month, week, day
df['DOJ'].dt.year
df['DOJ'].dt.month
df['DOJ'].dt.day
df['DOJ'].dt.day_name()
```

Out[585…
```
0      Monday
1      Monday
2    Thursday
3      Sunday
Name: DOJ, dtype: object
```

In [586…
```
# create new column using month extract function from DOJ column
df['Month'] = df['DOJ'].dt.month
```

In [587…
```
df
```

Out[587…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000	DA	2024-01-01	1
1	Raj	25	150000	DS	2024-01-15	1
2	Laksh	5	90000	CEO	2024-03-28	3
3	Advik	10	60000	HR	2024-03-03	3

In [588…
```
df['DOJ'] + pd.Timedelta(days=90) # add 90 days to DOJ column
```

Out[588…

```
0   2024-03-31
1   2024-04-14
2   2024-06-26
3   2024-06-01
Name: DOJ, dtype: datetime64[ns]
```

8. Handling Missing Values

In [589…

```
df
```

Out[589…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000	DA	2024-01-01	1
1	Raj	25	150000	DS	2024-01-15	1
2	Laksh	5	90000	CEO	2024-03-28	3
3	Advik	10	60000	HR	2024-03-03	3

In [590…

```
df.isnull()  # Detect missing values
```

Out[590…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False

In [591…

```
import numpy as np   # to create null values below
```

In [592…

```
df.loc[df.Name == 'Advik', 'Monthly_Salary'] = np.nan  # adding a null value
```

In [593…

```
df
```

Out[593…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000.0	DA	2024-01-01	1
1	Raj	25	150000.0	DS	2024-01-15	1
2	Laksh	5	90000.0	CEO	2024-03-28	3
3	Advik	10	NaN	HR	2024-03-03	3

In [594…

```
df.isnull()
```

Out[594…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	True	False	False	False

In [595…

```
# count of null value by columns

df.isnull().sum()
```

Out[595…

```
Name              0
Age               0
Monthly_Salary    1
Team              0
DOJ               0
Month             0
dtype: int64
```

In [596…

```
# fill null value with ZERO (0)

df.fillna(0)
```

Out[596…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000.0	DA	2024-01-01	1
1	Raj	25	150000.0	DS	2024-01-15	1
2	Laksh	5	90000.0	CEO	2024-03-28	3
3	Advik	10	0.0	HR	2024-03-03	3

In [597…

```
df.loc[df.Name=='Advik','Monthly_Salary'] = 60000
```

In [598…

```
df
```

Out[598…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000.0	DA	2024-01-01	1
1	Raj	25	150000.0	DS	2024-01-15	1
2	Laksh	5	90000.0	CEO	2024-03-28	3
3	Advik	10	60000.0	HR	2024-03-03	3

9. Aggregation and Group By

In [599…

```
df['Month'].value_counts()   # frequency of values in month column
```

Out[599…

```
Month
1    2
3    2
Name: count, dtype: int64
```

In [600…

```
df[df['Month']==1].value_counts()  # frequency of values in month column where m
```

Out[600…

```
Name    Age  Monthly_Salary  Team  DOJ         Month
Raj     25   150000.0        DS    2024-01-15  1        1
Satyam  22   100000.0        DA    2024-01-01  1        1
Name: count, dtype: int64
```

In [601…

```
# aggregation based on group by
df.groupby('Month')['Monthly_Salary'].sum()  # sum of salary by month
```

Out[601…

```
Month
1    250000.0
3    150000.0
Name: Monthly_Salary, dtype: float64
```

In [602…

```
# different aggregation on different columns

df.groupby('Month').agg({'Monthly_Salary' :'mean', 'Name' : 'count'})
```

Out[602…

	Monthly_Salary	Name
Month		
1	125000.0	2
3	75000.0	2

10. Concatenate & Merge DataFrame (JOINS)

In [603…

```
# Creating DataFrame
df1 = pd.DataFrame({'ID':[1,2,3],'Name':['A','B','C']})
df1
```

Out[603…

	ID	Name
0	1	A
1	2	B
2	3	C

In [604…

```
# Creating DataFrame
df2 = pd.DataFrame({'ID':[1,2,2,4],'Score':[88,96,77,79]})
df2
```

Out[604…

	ID	Score
0	1	88
1	2	96
2	2	77
3	4	79

Concatenate DataFrames (pd.concat)

Combine objects along a particular axis.

Concept :- ▪ Just joins DataFrames together ▪ No key matching ▪ Works row-wise or column-wise

In [605…

```
# Concatenate: vertical / row level / top on top

pd.concat([df1, df2], axis=0)
```

Out[605…

	ID	Name	Score
0	1	A	NaN
1	2	B	NaN
2	3	C	NaN
0	1	NaN	88.0
1	2	NaN	96.0
2	2	NaN	77.0
3	4	NaN	79.0

In [606…

```
# Concatenate: horizontal / column level / side on side

pd.concat([df1, df2], axis=1)
```

Out[606…

	ID	Name	ID	Score
0	1.0	A	1	88
1	2.0	B	2	96
2	3.0	C	2	77
3	NaN	NaN	4	79

Merge DataFrames (pd.merge)

Performs join operations similar to relational databases like SQL JOINS.

Concept :- ▪ Like SQL JOIN ▪ Combines based on common column(s)

In [607…

```
# merge = Join
pd.merge(df1,df2, how = 'inner', on= 'ID')
```

Out[607…

	ID	Name	Score
0	1	A	88
1	2	B	96
2	2	B	77

In [608…

```
pd.merge(df1, df2, how= 'inner', left_on='ID',right_on='ID')
```

Out[608…

	ID	Name	Score
0	1	A	88
1	2	B	96
2	2	B	77

11. Broadcasting

In [609…

```
df
```

Out[609…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	100000.0	DA	2024-01-01	1
1	Raj	25	150000.0	DS	2024-01-15	1
2	Laksh	5	90000.0	CEO	2024-03-28	3
3	Advik	10	60000.0	HR	2024-03-03	3

In [610…

```
df["Monthly_Salary"] = df["Monthly_Salary"] + 5000     # increasing salory 5000
df
```

Out[610…

	Name	Age	Monthly_Salary	Team	DOJ	Month
0	Satyam	22	105000.0	DA	2024-01-01	1
1	Raj	25	155000.0	DS	2024-01-15	1
2	Laksh	5	95000.0	CEO	2024-03-28	3
3	Advik	10	65000.0	HR	2024-03-03	3

12. Handling Duplicate Values

In [638…

```
# create data frame
df = pd.DataFrame({
    "name": ["A", "B", "A", "C"],
```

```
    "marks": [90, 80, 90, 70]
})
```

In [639…

```
df
```

Out[639…

	name	marks
0	A	90
1	B	80
2	A	90
3	C	70

In [640…

```
# show duplicate value
df[df.duplicated()]
```

Out[640…

	name	marks
2	A	90

In [ ]:

```
# to remove duplicates  (not executed)

df.drop_duplicates(keep="first")  # default
df.drop_duplicates(keep="last")
df.drop_duplicates(keep=False)    # remove all duplicates
```

In [641…

```
df.drop_duplicates()   # remove duplicate
```

Out[641…

	name	marks
0	A	90
1	B	80
3	C	70

13. Lambda Function

Lambda functions in pandas are anonymous one-line functions mainly used with apply, map, and groupby for quick transformations.

Use lambda for : ▪ Simple transformations ▪ Quick conditions ▪ Short logic

Avoid lambda when : ▪ Logic is complex → use def ▪ Needs reuse

In [643…

```
# apply ()

df["marks"] = df["marks"].apply(lambda x: x + 5)    # column-wise

df["result"] = df.apply(lambda row: row["marks"] * 2, axis=1)    # row-wise
```

In [645…

```
# map()

df["name"] = df["name"].map(lambda x: x.upper())
```

In [ ]:

```
# applymap()

df = df.applymap(lambda x: x * 2)
```

In [ ]:

```
# groupby()

df.groupby("name").agg(lambda x: x.max() - x.min())
```

In [ ]:

```
# Conditional Logic

df["grade"] = df["marks"].apply(
    lambda x: "Pass" if x >= 40 else "Fail"
)
```

NOTE :- all the above lambda methods are not executed

In [649…

```
# creating dataframe

df = pd.DataFrame({
    "name": ["A", "B", "C"],
    "marks": [35, 50, 90]
})
```

In [650…

```
df
```

Out[650…

	name	marks
0	A	35
1	B	50
2	C	90

In [651…

```
df["grade"] = df["marks"].apply(
    lambda x: "Pass" if x >= 40 else "Fail"
)   # using conditional logic
```

In [652…

```
df
```

Out[652…

	name	marks	grade
0	A	35	Fail
1	B	50	Pass
2	C	90	Pass

In [ ]: